

Bayesian Classifier for Sparsity-Promoting Feature Selection

Danlei Xu*, Lan Du[†], Hongwei Liu[‡] and Penghui Wang[§]

The National Laboratory of Radar Signal Processing

Xidian University, Xi'an, China

**xdlei5258@163.com*

[†]dulan@mail.xidian.edu.cn

[‡]hwliu@xidian.edu.cn

[§]wangpenghui@mail.xidian.edu.cn

Received 20 November 2013

Accepted 9 June 2015

Published 3 August 2015

A Bayesian classifier for sparsity-promoting feature selection is developed in this paper, where a set of nonlinear mappings for the original data is performed as a pre-processing step. The linear classification model with such mappings from the original input space to a nonlinear transformation space can not only construct the nonlinear classification boundary, but also realize the feature selection for the original data. A zero-mean Gaussian prior with Gamma precision and a finite approximation of Beta process prior are used to promote sparsity in the utilization of features and nonlinear mappings in our model, respectively. We derive the Variational Bayesian (VB) inference algorithm for the proposed linear classifier. Experimental results based on the synthetic data set, measured radar data set, high-dimensional gene expression data set, and several benchmark data sets demonstrate the aggressive and robust feature selection capability and comparable classification accuracy of our method comparing with some other existing classifiers.

Keywords: Feature selection; sparsity; classifier; beta process; relevance vector machine (RVM); variational Bayesian (VB) inference.

1. Introduction

Advances in data collection and storage capabilities during the past decades have led to an information overload.¹⁵ High-dimensional data sets present many mathematical challenges as well as some opportunities. One of the problems is that, in many cases, not all of the measured variables are important. Thus reducing the dimension of the original data is of interest in many applications. Generally, traditional classification schemes first reduce the original data dimension and then put the

[†] Corresponding author.

dimensionality-reduced samples into the classifiers. However, there may be mismatching between the dimensionality reduction method and the classifier model. Therefore, to avoid such problem, we plan to design a classifier which can achieve the dimensionality reduction and the data classification simultaneously.

In the binary learning problems, we are given a data set of input-label pairs $\{\mathbf{x}_n, t_n\}_{n=1}^N$ with $n \in \{1, 2, \dots, N\}$.^a Assume that the input sample is a P -dimensional feature vector $\mathbf{x}_n = [x_{1n}, \dots, x_{in}, \dots, x_{Pn}] \in \mathbb{R}^{1 \times P}$ with $i \in \{1, 2, \dots, P\}$, and $t_n \in \{1, -1\}$ is the label associated with the feature vector $\mathbf{x}_n$. We wish to learn a function to describe the dependence of the labels on the inputs with the objective of making accurate predictions of t_* for a new sample $\mathbf{x}_*$ (known as classifier design), and to identify a subset of features from the high-dimensional feature vector that is most discriminative about the class distinction (known as feature selection). In this paper, we develop a Bayesian classifier for sparsity-promoting feature selection (referred to as BCSFS in this paper) to accomplish both classification and feature selection simultaneously.

Tipping proposed the Relevance Vector Machine (RVM),^{4,34} a probabilistic model whose functional form is equivalent to the Support Vector Machine (SVM),⁶ i.e. $y(\mathbf{x}, \mathbf{w}) = \sum_{n=1}^N w_n K(\mathbf{x}, \mathbf{x}_n) + w_0$, where $\{w_n\}_{n=0}^N$ are the weights and $K(\mathbf{x}, \mathbf{x}_n)$ is a kernel function. Two of the most popular kernels are the r th order polynomial and the Gaussian Radial Basis Function (RBF).²⁴ The RVM achieves comparable recognition accuracy to the SVM, yet provides a full predictive distribution, and also usually requires fewer relevance/support vectors.^{4,34} Nevertheless, neither of the traditional RVM and SVM has the ability to select features, which may affect classification performance for a data set containing a large number of irrelevant and redundant variables.

To realize feature selection, the RVM in no kernel function form was adopted,^{23,25} which can impose sparsity in the input space. Similarly, l_1 -norm sparse SVM is proposed, which uses $\|\cdot\|_1$ as the regularizer.^{5,17,36} The optimization is convex, and can be solved optimally by Linear Programming (LP) solvers or Newton method.¹⁷ Except for l_1 -norm sparse SVM, an approximation of the zero norm minimization is proposed to solve sparse SVM with $\|\cdot\|_0$ as the regularizer, namely l_0 -norm sparse SVM.³⁵ However, the problem is nonconvex and may suffer from local minima. Recently, by introducing a 0-1 control variable to each input feature, l_0 -norm sparse SVM was converted to a Mixed Integer Programming (MIP) problem.³¹ Rather than directly solving this MIP, it proposed an efficient cutting plane algorithm combining with multiple kernel learning to solve its convex relaxation.³¹ Nevertheless, the number of the sparse features needs to be given in advance.³¹

The linear regression model $y(\mathbf{x}_n, \mathbf{w}) = \sum_{i=1}^P w_i x_{in}$ for feature selection has been widely used in many methods.^{14,21,23,25,27} In the classification problem, $y(\mathbf{x}_n, \mathbf{w})$ is a continuous variable hidden behind the label t_n , and the link function between

^aThroughout this paper normal lowercase symbols represent scales, bold lowercase symbols denote vectors, and capital bold symbols denote matrices.

$y(\mathbf{x}_n, \mathbf{w})$ and t_n can be the logistic function^{21,27} or Probit function.¹⁴ To realize feature selection, the l_1 -norm constraint,²¹ the l_p -norm constraint ($\tilde{p} < 1$),²⁷ or the Jeffreys' prior¹⁴ can be used. The model parameters can be estimated via different approaches, mainly including the optimization methods^{21,27} and expectation-maximization (EM) approach.¹⁴ In addition, there is another shrinkage and selection method, lasso,^{16,32,33} is also suitable for this linear regression. It minimizes the usual sum of squared errors, with a bound on the sum of the absolute values of the coefficients. Nevertheless, since the regression model is linear, all these classifiers can just construct the linear classification boundary, which may not guarantee the good performance for some nonlinear or complicated classification problems.

A Joint Classifier and Feature Optimization (JCFO) method²⁴ is proposed, which is a Bayesian approach to simultaneously learn both an optimal nonlinear classifier and a subset of features that is most relevant to the classification task. JCFO uses two heavy-tailed priors, one to promote sparsity in the utilization of basis functions and the other to realize sparsity in feature utilization. However, from the results in Sec. 4,²⁴ we can see that the selected features via JCFO are not very sparse (i.e. more than half of all features are selected for the benchmark data sets), which means the first sparse parameter for selecting basis functions dominates this model.

This paper seeks a linear Bayesian classifier to realize feature selection via a set of nonlinear mappings from the original input space to a nonlinear transformation space, where both features and nonlinear mapping functions are put on sparsity-promoting priors. For the optimization frame, e.g. SVM and SVM variant methods, a tuning parameter is necessary to control the tradeoff between loss and penalty, whose values have great influence on the performance.⁶ However, for the nonparametric Bayesian frame, it has less sensitivity to the parameters by setting the reasonable prior distributions. According to the Bayesian theory, more accurate result can be obtained via the reasonable prior distribution setting. In addition, compared with the point estimate, the Bayesian formalism provides a full posterior density function for the parameters.³ To avoid one sparse prior dominate over the other one as in JCFO, the two variables with sparse priors construct the classification boundary together, thus both priors can promote strong sparsity in this model. Unlike in Ref. 31 the number of selected features must be given firstly, our method can determine it automatically. In addition, in this paper BCSFS is formulated and solved within a completely Bayesian paradigm via the variational inference, thereby giving a posterior distribution over latent variables.

The remainder of the paper is organized as follows: In Sec. 2, we introduce our linear Bayesian classifier with the discussion about the used Bayesian priors. We derive a variational Bayesian (VB) inference algorithm to get the posterior distributions on the latent variables in Sec. 3. Experimental results on the synthetic data set, measured radar data set, high-dimensional gene data set, and some benchmark data sets are reported in Sec. 4. Section 5 summarizes the paper by presenting some concluding remarks.

2. Bayesian Classifier for Sparsity-Promoting Feature Selection

2.1. Basic model construction

Before considering the classification problem, we first introduce the regression problem for easy understanding. For the regression problem, we assume the training set is $\{\mathbf{x}_n, y_n\}_{n=1}^N$, where y_n is a continuous variable. In this paper, we consider the regression function that is linear with respect to the parameter vector $\mathbf{w}$, thus the basic construction of our linear regression model³ is as follows:

$$y_n = (\Phi(\mathbf{x}_n) \bullet (\mathbf{w} \odot \mathbf{z})) + \varepsilon_n = \sum_{i=1}^P (\phi(x_{in}) \bullet (\mathbf{w}_i \odot \tilde{\mathbf{z}}_i)) + \varepsilon_n, \quad (1)$$

where the symbol $\odot$ represents the Hadamard, or element-wise multiplication of two vectors, $\Phi(\mathbf{x}_n) = [\phi(x_{1n}), \dots, \phi(x_{in}), \dots, \phi(x_{Pn})] \in \mathbb{R}^{1 \times MP}$ is a set of nonlinear mappings of the input sample $\mathbf{x}_n$, wherein $\phi(x_{in}) = [g(x_{in}, 1), \dots, g(x_{in}, m), \dots, g(x_{in}, M)] \in \mathbb{R}^{1 \times M}$ is the nonlinear transformations of the i th feature, with $m \in \{1, \dots, M\}$ and usually $M \geq 1$. The three commonly used mapping functions designed in our paper take the forms:

power transformation:

$$g(x_{in}, m) = x_{in}^m, \quad (2)$$

exponent transformation:

$$g(x_{in}, m) = \exp(x_{in}/m), \quad (3)$$

sine transformation:

$$g(x_{in}, m) = \sin(x_{in}/m), \quad (4)$$

the mapping functions here directly formulate the transformations of the original data, while a kernel function formulates an inner production of the unknown mapping in the transformed space.³ Due to the extension of the input sample, the corresponding weight is also extended as $\mathbf{w} = [\mathbf{w}_1, \dots, \mathbf{w}_i, \dots, \mathbf{w}_P]^T \in \mathbb{R}^{MP \times 1}$ with $\mathbf{w}_i = [w_{i1}, \dots, w_{im}, \dots, w_{iM}]$. The expression $(\Phi(\mathbf{x}_n) \bullet (\mathbf{w} \odot \mathbf{z}))$ in (1) represents the inner product between the mapping functions $\Phi(\mathbf{x}_n)$ and the weight vector $(\mathbf{w} \odot \mathbf{z})$. As mentioned above, $\Phi(\mathbf{x}_n)$ is a nonlinear transformation of the input sample $\mathbf{x}_n$, thus our model is different from the model $y(\mathbf{x}_n, \mathbf{w}) = \sum_{i=1}^P w_i x_{in}$, which imposes sparsity in the input space without the nonlinear transformation. Here we emphasize the characteristic of our model, i.e. the “nonlinear feature selection” (selecting the discriminating features for the nonlinear or complicated classification problems) in the original input space, to distinguish our model with the traditional linear feature selection.^{14,21,23–25,27,31} Nevertheless, not all of the nonlinear transformations are indispensable (some might be redundant during describing the dependence

of the targets on the inputs). Therefore, to avoid severe overfitting with too many parameters corresponding to the nonlinear transformations, we seek sparsity in the utilization of nonlinear transformations. Thus $\mathbf{z} = [\tilde{z}_1, \dots, \tilde{z}_i, \dots, \tilde{z}_P]^T \in \mathbb{R}^{MP \times 1}$ is adopted here, which is a binary vector with $\tilde{z}_i = [z_1, \dots, z_m, \dots, z_M]$ and $z_m \in \{0, 1\}$. Obviously, setting $z_m = 0$ is equivalent to entirely removing all the m th nonlinear transformed data $\{g(x_{in}, m)\}_{i=1}^P$ from $\Phi(\mathbf{x}_n)$.

In Eq. (1), ε_n is the independent noise which is assumed to be zero-mean Gaussian distributed with inverse Gamma variance τ_n^{-1} ,³⁴ i.e. $\varepsilon_n \sim N(0, \tau_n^{-1})$ and $\tau_n \sim \text{Gamma}(\tau_n | c, d)$, where the notation $N(\cdot)$ specifies the Gaussian distribution and $\text{Gamma}(\cdot)$ represents the Gamma distribution. Thus $p(y_n | \mathbf{x}_n) = N(y_n | (\Phi(\mathbf{x}_n) \cdot (\mathbf{w} \odot \mathbf{z})), \tau_n^{-1})$.

After briefly introducing the regression model, we now go back to the binary classification problem. As described in Sec. 1, for the classification problem, the training set is $\{\mathbf{x}_n, t_n\}_{n=1}^N$, t_n is of categorical nature (e.g. $t_n \in \{1, -1\}$). In the classification problem, the formulation is somewhat more complicated due to the categorical nature of the output variable.¹⁴ A standard approach is to consider a generalized linear model (GLM).²⁸ For a binary classification problem, GLM models the probability that an observation $\mathbf{x}_n$ belongs to class 1, as given by a nonlinear function applied to the output of the linear regression function $y_n = (\Phi(\mathbf{x}_n) \cdot (\mathbf{w} \odot \mathbf{z})) + \varepsilon_n$. Formally, let this nonlinear function (called the activation function or link function²⁸) be denoted as $\psi: \mathbb{R} \rightarrow [0, 1]$; then the GLM is defined as $p(t = 1 | \mathbf{x}_n) = \psi((\Phi(\mathbf{x}_n) \cdot (\mathbf{w} \odot \mathbf{z})) + \varepsilon_n)$.

Although the most common link between regression and classification is the Logistic function $\psi(y) = (1 + \exp(-y))^{-1}$,¹⁴ here we adopt the Probit link^{7-10,19,30} as the basic model construction. The logistic link function, though easy to understand, is not a standard probabilistic function and induces some difficulties in the inference procedure. In addition, the fundamental reasons behind our choice of the Probit link are its simple interpretation in terms of hidden variables¹⁴ and its general extension to multiclass classifier via multinomial Probit link.^{7-10,19,30}

To extend regression model directly to binary classification model, we give the probability relation between t_n and y_n :

$$p(t_n | y_n) = I(t_n = \text{sign}(y_n)) = \begin{cases} 1 & t_n = \text{sign}(y_n), \\ 0 & \text{otherwise,} \end{cases} \quad (5)$$

where $I(\cdot)$ is an indicator function, $\text{sign}(y_n) = 1$ when $y_n \geq 0$ and $\text{sign}(y_n) = -1$ when $y_n < 0$. For the classification problem, y_n is a continuous variable hidden behind the label t_n , which can also be regarded as the output of the regression problem.

Therefore, the model for binary classification is:

$$p(t_n, y_n | (\Phi(\mathbf{x}_n) \cdot (\mathbf{w} \odot \mathbf{z})), \tau_n) = p(t_n | y_n) N(y_n | (\Phi(\mathbf{x}_n) \cdot (\mathbf{w} \odot \mathbf{z})), \tau_n^{-1}). \quad (6)$$

By integrating out y_n , we can obtain the Probit link:

$$\begin{aligned} p(t_n = 1 \mid (\Phi(\mathbf{x}_n) \bullet (\mathbf{w} \odot \mathbf{z})), \tau_n) &= \int_0^\infty N(y_n \mid (\Phi(\mathbf{x}_n) \bullet (\mathbf{w} \odot \mathbf{z})), \tau_n^{-1}) dy_n \\ &= \text{normcdf}((\Phi(\mathbf{x}_n) \bullet (\mathbf{w} \odot \mathbf{z})) / (\tau_n^{-1/2})), \end{aligned} \quad (7)$$

where $\text{normcdf}(\cdot)$ represents the Normal cumulative distribution function.

2.2. Sparsity-promoting priors

Sparseness is desirable in the supervised learning since it leads to a structural simplification of the model.¹⁴ The sparseness of a variable is defined as that the irrelevant or redundant components are exactly zero or tend to be zero.

Our classifier seeks sparsity in its use of both features and nonlinear mapping functions.

To favor a feature-sparseness, a zero-mean Gaussian prior can be adopted for $\mathbf{w}$,³⁴ that is:

$$p(\mathbf{w} \mid \boldsymbol{\alpha}) = \prod_{i=1}^P \prod_{m=1}^M N(w_{im} \mid 0, \alpha_i^{-1}), \quad (8)$$

where $\boldsymbol{\alpha} = [\boldsymbol{\alpha}_1, \dots, \boldsymbol{\alpha}_i, \dots, \boldsymbol{\alpha}_P]^T$ with $\boldsymbol{\alpha}_i = [\alpha_i, \alpha_i, \dots, \alpha_i]_{1 \times M}$ for $i = 1, \dots, P$. To complete the specification of this hierarchical prior, Gamma hyperpriors are placed on $\boldsymbol{\alpha}$ ³⁴:

$$p(\boldsymbol{\alpha}) = \prod_{m=1}^M \prod_{i=1}^P \text{Gamma}(\alpha_i \mid a, b) = \prod_{m=1}^M \prod_{i=1}^P \Gamma(a)^{-1} b^a \alpha_i^{a-1} e^{-b\alpha_i}, \quad (9)$$

with $\Gamma(a) = \int_0^\infty t^{a-1} e^{-t} dt$ denoting the gamma function. It has been demonstrated that appropriate choice of parameters a and b encourages a sparse representation for the coefficients in the vector $\mathbf{w}$.³⁴ Typically, when $a = b = \varsigma$, with $\varsigma > 0$ a small positive constant (e.g. $\varsigma = 10^{-6}$ in the following experiments), $\text{Gamma}(\alpha_i \mid a, b)$ has a large spike concentrated at zero and a heavy right tail. The spike corresponds to the most discriminative or informative features. At the other extreme, features whose α_i is in the right tail have coefficients $\{w_{im}\}_{m=1}^M$ that are shrunk strongly to zero, favoring sparseness, while borrowing information about which features are not important for the classification task.²⁰

The above hierarchical priors are similar to those in the RVM, but the difference is that here $\boldsymbol{\alpha}$ is also extended with the nonlinear transformations. Notice that, here $\{w_{im}\}_{m=1}^M$ for each i can vary across the M nonlinear transformations, which means that different nonlinear transformations of the same feature can play different roles in constructing the classification boundary. However, we expect α_i is set to be fixed for different nonlinear transformations of the i th feature, i.e. α_i are shared among all M nonlinear transformed data. This means that all M nonlinear transformations $\{g(x_{in}, m)\}_{m=1}^M$ of each feature x_{in} are utilized to learn α_i , and the feature x_{in} will be

selected or not selected for all M transformations. This characteristic is one advantage of our method: all M nonlinear transformations information of each feature x_{in} is utilized to determine whether x_{in} is selected or not, which is different from some traditional feature selection methods^{14,21,23–25,27,31,36} with only x_{in} or one form of its nonlinear transformations used for feature selection. Therefore, the proposed prior structure can promote the sparse nonlinear feature selection, and form the nonlinear classification boundary for the linear model as well. Here please note the deterministic parameters for selecting features are $\{\alpha_i\}_{i=1}^P$, whose number is still P .

For the binary vector $\mathbf{z}$, we use the Bernoulli-Beta prior, which is a finite approximation of the Beta process²⁹:

$$p(\mathbf{z} | \mathbf{u}) = \prod_{m=1}^M \prod_{i=1}^P \text{Bernoulli}(z_m | u_m) = \prod_{m=1}^M \prod_{i=1}^P u_m^{z_m} (1 - u_m)^{1-z_m}, \quad (10)$$

$$p(\mathbf{u}) = \prod_{m=1}^M \prod_{i=1}^P \text{Beta}(u_m | e/K, f(K-1)/K), \quad (11)$$

where $\text{Bernoulli}(\cdot)$ and $\text{Beta}(\cdot)$ refer respectively to Bernoulli and Beta densities parameterized by the arguments inside parentheses, $\mathbf{u} = [\tilde{\mathbf{u}}_1, \dots, \tilde{\mathbf{u}}_i, \dots, \tilde{\mathbf{u}}_P]^T \in \mathbb{R}^{MP \times 1}$ with $\tilde{\mathbf{u}}_i = [u_1, \dots, u_m, \dots, u_M]$ and hyperparameters $e, f > 0$. Usually, we set K to a large, but finite number. Therefore, the expectation of u_m , i.e. $\langle u_m \rangle = (e/K)/(e/K + f(K-1)/K)$, is close to zero, which means $p(z_m = 1)$ is very small and favors sparsity in $\mathbf{z}$ (most elements are 0). In this way, the proposed prior structure can promote to avoiding overfitting and constructing the simplest classification boundary for a given data set.

According to the priors described above, BCSFS promotes sparsity in its use of both features (sparsity in $\boldsymbol{\alpha} = [\boldsymbol{\alpha}_1, \dots, \boldsymbol{\alpha}_i, \dots, \boldsymbol{\alpha}_P]^T$) and nonlinear mappings (sparsity in $\mathbf{z} = [\tilde{\mathbf{z}}_1, \dots, \tilde{\mathbf{z}}_i, \dots, \tilde{\mathbf{z}}_P]^T$). The construction of BCSFS is different from that of JCFO. In JCFO, sparsity in the use of basis functions is put in the kernel space to construct the classification boundary, whereas sparsity in the use of features is put in the original input space to select features. Since the two sparsity-promoting parameters are put in the different spaces for different purposes in JCFO, the sparse parameter for selecting basis functions dominates this model and the selected features are not very sparse. However, in BCSFS, the sparsities in the use of features and nonlinear mappings are both put in the original input space to construct the classification boundary together, where the feature selection is directly connected with the classification boundary construction, thus both of the sparsity-promoting parameters can promote strong sparsity in this model.

Through the aforementioned introduction of the model and the detailed parameters, we now have a complete probabilistic specification of this model, with a graphical model representation illustrated in Fig. 1. In the figure, the shaded and unshaded circular nodes indicate the observed and latent variables, respectively; and the square nodes indicate the hyperparameters. An arrow indicates the dependence

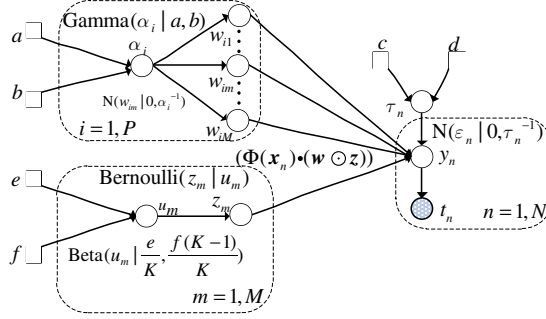


Fig. 1. Graphical representation of the proposed BCSFS model. The detailed variables are introduced in Sec. 2.

between the variables. The boxes denote the repetition, with the number of repetitions represented by the variables in the corner of boxes.²⁶ This structural form is very convenient for implementing the iterative algorithm for evaluation of the posterior density functions for w_i , z_m , and τ_n .

3. VB Inference

The proposed BCSFS model may be readily implemented via Markov Chain Monte Carlo (MCMC) sampling.^{8,11,18,26} However, we employ VB^{2,7,9,11–13} inference in this paper, as a compromise between accuracy and efficiency.

Bayesian inference seeks to estimate the posterior distributions of the latent variables Ψ in the model, given the observed data D and hyperparameters θ .

$$p(\Psi | D, \theta) = p(D | \Psi, \theta) p(\Psi | \theta) / \left(\int p(D | \Psi, \theta) p(\Psi | \theta) d\Psi \right), \quad (12)$$

where the denominator $\int p(D | \Psi, \theta) p(\Psi | \theta) d\Psi = p(D | \theta)$ is the model evidence (marginal likelihood). VB inference approximates an intractable joint posterior $p(\Psi | D, \theta)$ of all the hidden variables by a product of marginal distributions $q(\Psi) = \prod_f q(\Psi_f)$, each over only a single latent variable Ψ_f . The log marginal likelihood for the model is:

$$\ln p(D | \theta) = \text{KL}(q(\Psi) | p(\Psi | D, \theta)) + L(q(\Psi)), \quad (13)$$

where $\text{KL}(q(\Psi) | p(\Psi | D, \theta))$ is Kullback–Leibler divergence between the variational approximation $q(\Psi)$ and the true joint posterior $p(\Psi | D, \theta)$. Since $\text{KL}(q(\Psi) | p(\Psi | D, \theta)) \geq 0$, it follows that $L(q(\Psi))$ is a rigorous lower bound on $\ln p(D | \theta)$. Accordingly, the optimal parameterization of $q(\Psi_f)$ is obtained by minimizing $\text{KL}(q(\Psi) | p(\Psi | D, \theta))$, i.e. maximizing the similarity between the variational distribution and the true posterior.

In the BCSFS model, D is $\{\mathbf{x}_n, t_n\}_{n=1}^N$, Ψ is $\{y_n, w_{im}, \alpha_i, \tau_n, z_m, u_m\}_{n=1}^N \{m=1}^M \{i=1}^P$, and θ is $\{a, b, c, d, e, f\}$. The VB inference for the model is summarized in Algorithm 1, which starts from a set of initial values θ_0 , and when the lower bound

Algorithm 1: VB-based Inference for BCSFS

Input: The input-label pairs $\mathbf{D} = \{\mathbf{x}_n, t_n\}_{n=1}^N$; the initial hyper-parameters $\boldsymbol{\theta}_0 = \{a, b, c, d, e, f\}$; a small positive constant ς .

Output: The posterior distributions of the latent variables Ψ : $q(y_n)$, $q(\mathbf{w})$, $q(\boldsymbol{\alpha})$, $q(\tau_n)$, $q(z_m)$ and $q(u_m)$; the lower bound L .

Initialization: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}_0$; iter = 0; change = 1; and $L(1) = \int q(\Psi) \ln \frac{p(\mathbf{D}, \Psi)}{q(\Psi)} d\Psi$.

While change $> \varsigma$

 iter=iter+1

$$\begin{aligned} q(y_n) &= N^{t_n}(\tilde{m}_n, \tilde{\sigma}_n), \\ \tilde{m}_n &= (\Phi(\mathbf{x}_n) \bullet (\langle \mathbf{w} \rangle \odot \langle \mathbf{z} \rangle)), \tilde{\sigma}_n = \langle \tau_n \rangle^{-1}, \end{aligned} \quad (14)$$

$$\begin{aligned} q(\mathbf{w}) &= N(\tilde{\boldsymbol{\mu}}_w, \tilde{\boldsymbol{\Sigma}}_w), \\ \tilde{\boldsymbol{\Sigma}}_w &= (\text{diag}(\boldsymbol{\alpha}) + (\Phi(\mathbf{X})^T \text{diag}(\langle \boldsymbol{\tau} \rangle) \Phi(\mathbf{X})) \odot (\langle \mathbf{z} \mathbf{z}^T \rangle))^{-1}, \\ \tilde{\boldsymbol{\mu}}_w &= \tilde{\boldsymbol{\Sigma}}_w ((\langle \mathbf{z}_N \rangle \odot \Phi(\mathbf{X})^T) \text{diag}(\langle \boldsymbol{\tau} \rangle \langle \mathbf{y} \rangle)), \end{aligned} \quad (15)$$

$$\begin{aligned} q(\alpha_i) &= \text{Gamma}(\tilde{a}_i, \tilde{b}_i), \quad \tilde{a}_i = a_i + \frac{1}{2}, \quad \tilde{b}_i = b_i + \frac{1}{M} \sum_{m=1}^M \frac{1}{2} \langle w_{im}^2 \rangle, \\ q(\tau_n) &= \text{Gamma}(\tilde{c}_n, \tilde{d}_n), \quad \tilde{c}_n = c_n + \frac{1}{2}, \end{aligned} \quad (16)$$

$$\begin{aligned} \tilde{d}_n &= d_n + \frac{1}{2} (\langle y_n^2 \rangle - 2 \langle y_n \rangle (\Phi(\mathbf{x}_n) \bullet (\langle \mathbf{w} \rangle \odot \langle \mathbf{z} \rangle))) \\ &\quad + \Phi(\mathbf{x}_n) (\langle \mathbf{w} \mathbf{w}^T \rangle \odot (\langle \mathbf{z} \mathbf{z}^T \rangle)) \Phi(\mathbf{x}_n)^T, \end{aligned} \quad (17)$$

$$q(z_m) = \text{Bernoulli} \left(\frac{p(z_m = 1)}{p(z_m = 0) + p(z_m = 1)} \right), \quad (18)$$

$$\begin{aligned} q(u_m) &= \text{Beta} \left(\frac{\tilde{e}_m}{K}, \frac{\tilde{f}_m(K-1)}{K} \right), \quad \tilde{e}_m = e_m + K \langle z_m \rangle, \\ \tilde{f}_m &= f_m + \frac{K}{K-1} (1 - \langle z_m \rangle), \end{aligned} \quad (19)$$

$$L(\text{iter} + 1) = \int q(\Psi) \ln \frac{p(\mathbf{D}, \Psi)}{q(\Psi)} d\Psi, \quad (20)$$

$$\text{change} = (L(\text{iter} + 1) - L(\text{iter})) / L(\text{iter}), \quad (21)$$

End

variation is lower than a small positive constant ς (we set $\varsigma = 10^{-6}$), we terminate the algorithm.

In Algorithm 1, $\langle \cdot \rangle$ denotes the posterior expectation for the variable over the corresponding distribution on it. In Eq. (26), the probability of y_n is shown to be truncated normal¹ (i.e. $N^{t_n}(\cdot)$, the direction of truncation is determined by t_n), therefore, for $N^{t_n}(y_n; (\Phi(\mathbf{x}_n) \cdot (\langle \mathbf{w} \rangle \odot \langle \mathbf{z} \rangle)), \langle \tau_n \rangle^{-1})$, we can calculate the expectations of y_n and y_n^2 :

$$\begin{aligned} \langle y_n \rangle &= (\Phi(\mathbf{x}_n) \cdot (\langle \mathbf{w} \rangle \odot \langle \mathbf{z} \rangle)) + t_n \frac{1}{\langle \tau_n \rangle^{\frac{1}{2}}} \\ &\times \frac{\text{normpdf}((\Phi(\mathbf{x}_n) \cdot (\langle \mathbf{w} \rangle \odot \langle \mathbf{z} \rangle)) \langle \tau_n \rangle^{\frac{1}{2}})}{\text{normcdf}(t_n(\Phi(\mathbf{x}_n) \cdot (\langle \mathbf{w} \rangle \odot \langle \mathbf{z} \rangle)) \langle \tau_n \rangle^{\frac{1}{2}})}, \end{aligned} \quad (22)$$

$$\begin{aligned} \langle y_n^2 \rangle &= \frac{1}{\langle \tau_n \rangle} + (\Phi(\mathbf{x}_n) \cdot (\langle \mathbf{w} \rangle \odot \langle \mathbf{z} \rangle))^2 \\ &+ t_n (\Phi(\mathbf{x}_n) \cdot (\langle \mathbf{w} \rangle \odot \langle \mathbf{z} \rangle)) \frac{1}{\langle \tau_n \rangle^{\frac{1}{2}}} \frac{\text{normpdf}((\Phi(\mathbf{x}_n) \cdot (\langle \mathbf{w} \rangle \odot \langle \mathbf{z} \rangle)) \langle \tau_n \rangle^{\frac{1}{2}})}{\text{normcdf}(t_n(\Phi(\mathbf{x}_n) \cdot (\langle \mathbf{w} \rangle \odot \langle \mathbf{z} \rangle)) \langle \tau_n \rangle^{\frac{1}{2}})}, \end{aligned} \quad (23)$$

where $\text{normpdf}(\cdot)$ represents the Normal probability density function. In Eq. (15) for updating $\mathbf{w}$, $\Phi(\mathbf{X}) = [\Phi(\mathbf{x}_1)^T, \Phi(\mathbf{x}_2)^T, \dots, \Phi(\mathbf{x}_N)^T]^T$ and $\mathbf{z}_N$ is a matrix consisting of a 1-by- N tilling of copies of $\mathbf{z}$. When update for $\mathbf{z}$ in Eq. (18), the posterior of z_m is $q(z_m) = \text{Bernoulli}(\frac{p(z_m=1)}{p(z_m=0)+p(z_m=1)})$ with the probability that $z_m = 1$ proportional to

$$\begin{aligned} p(z_m = 1) &\propto \exp(\langle \ln(u_m) \rangle) \times \exp \left(\frac{1}{P} \sum_{n=1}^N \left(-\frac{\langle \tau_n \rangle}{2} \left(-2\langle y_n \rangle \sum_{i=1}^P w_{im} g(x_{in}, m) \right. \right. \right. \\ &\quad \left. \left. \left. + \Phi(\mathbf{x}_n) (\langle \mathbf{w} \mathbf{w}^T \rangle \odot \mathbf{z}_{z_m=1}^T \mathbf{z}_{z_m=1}^T) \Phi(\mathbf{x}_n)^T \right) \right) \right) \end{aligned} \quad (24)$$

and the probability that $z_m = 0$ proportional to

$$\begin{aligned} p(z_m = 0) &\propto \exp(\langle \ln(1 - u_m) \rangle) \times \exp \left(\frac{1}{P} \sum_{n=1}^N \left(-\frac{\langle \tau \rangle}{2} \Phi(\mathbf{x}_n) (\langle \mathbf{w} \mathbf{w}^T \rangle \right. \right. \\ &\quad \left. \left. \odot \mathbf{z}_{z_m=0}^T \mathbf{z}_{z_m=0}^T) \Phi(\mathbf{x}_n)^T \right) \right), \end{aligned} \quad (25)$$

In Eq. (24), $\mathbf{z}_{z_m=1}$ is obtained from $\langle \mathbf{z} \rangle$ with the elements corresponding to z_m replaced by 1, $\mathbf{z}_{z_m=0}$ is defined similarly to $\mathbf{z}_{z_m=1}$ in Eq. (25). In addition, in Eqs. (24) and (25), $\langle \ln(u_m) \rangle$ and $\langle \ln(1 - u_m) \rangle$ are used, which can be calculated according to

the update of u_m in Eq. (19):

$$\begin{aligned}\langle \ln(u_m) \rangle &= \vartheta(\tilde{e}_m/K) - \vartheta((\tilde{e}_m + \tilde{f}_m(K-1))/K), \\ \langle \ln(1-u_m) \rangle &= \vartheta(\tilde{f}_m(K-1)/K) - \vartheta((\tilde{e}_m + \tilde{f}_m(K-1))/K),\end{aligned}\quad (26)$$

where $\vartheta(\cdot)$ represents the digamma function.

After the VB algorithm for parameter estimation converges, we can make the prediction for a test sample $\mathbf{x}_*$ based on the posterior distribution over the weights and conditioned on the mean values $\langle \mathbf{z} \rangle$ and $\langle \tau_* \rangle$, where $\langle \tau_* \rangle = \frac{1}{N} \sum_{n=1}^N \langle \tau_n \rangle$. Then the predictive distribution can be calculated as:

$$\begin{aligned}p(t_* = 1 | \mathbf{x}_*) &= \int_0^\infty N(y; (\Phi(\mathbf{x}_*) \bullet (\tilde{\boldsymbol{\mu}}_w \odot \langle \mathbf{z} \rangle)), (\Phi(\mathbf{x}_*) \odot \langle \mathbf{z} \rangle)^T) \\ &\quad \times \tilde{\boldsymbol{\Sigma}}_w(\Phi(\mathbf{x}_*)^T \odot \langle \mathbf{z} \rangle) + \langle \tau_* \rangle^{-1} dy \\ &= \text{normcdf} \left(\frac{(\Phi(\mathbf{x}_*) \bullet (\tilde{\boldsymbol{\mu}}_w \odot \langle \mathbf{z} \rangle))}{\sqrt{(\Phi(\mathbf{x}_*) \odot \langle \mathbf{z} \rangle)^T \tilde{\boldsymbol{\Sigma}}_w(\Phi(\mathbf{x}_*)^T \odot \langle \mathbf{z} \rangle) + \langle \tau_* \rangle^{-1}}} \right).\end{aligned}\quad (27)$$

According to the predictive distribution, we can classify the test sample. If $p(t_* = 1 | \mathbf{x}_*) \geq 0.5$, the test sample is determined as one class with label 1, otherwise, it is recognized as the other class with label -1 . Of course, we can also classify the test sample according to the predictive mean $(\Phi(\mathbf{x}_*) \bullet (\tilde{\boldsymbol{\mu}}_w \odot \langle \mathbf{z} \rangle))$ as the RVM does.³⁴ In addition, after the algorithm converges, we can determine which features and nonlinear transformations are selected by observing the values of $\{\langle \alpha_i \rangle = \tilde{a}_i/\tilde{b}_i\}_{i=1}^P$ and $\{\langle z_m \rangle\}_{m=1}^M$.

4. Experimental Results

In this section, we first present some visualizations of BCSFS applied to simple example of synthetic data set to demonstrate its capabilities of feature selection, nonlinear transformation selection and classification, followed by experimental results on measured radar data set, the high-dimensional gene data set, and several benchmark data sets. To obtain the meaningful comparisons with other algorithms, the same training samples and test samples are fixed for each data set. All samples in each data set are initially normalized to have zero mean and unit variance.

4.1. Results on Ripley's synthetic data

The effectiveness of BCSFS on synthetic data is evaluated in this part. Our goal is to examine whether our method can choose the features that are informative about the class distinction and the indispensable nonlinear mapping functions that can construct the decision boundary for the linear model, as well as obtain the comparable classification performance. In addition, the comparative experiments between VB approximation and the full MCMC Gibbs sampling method are also given in this part.

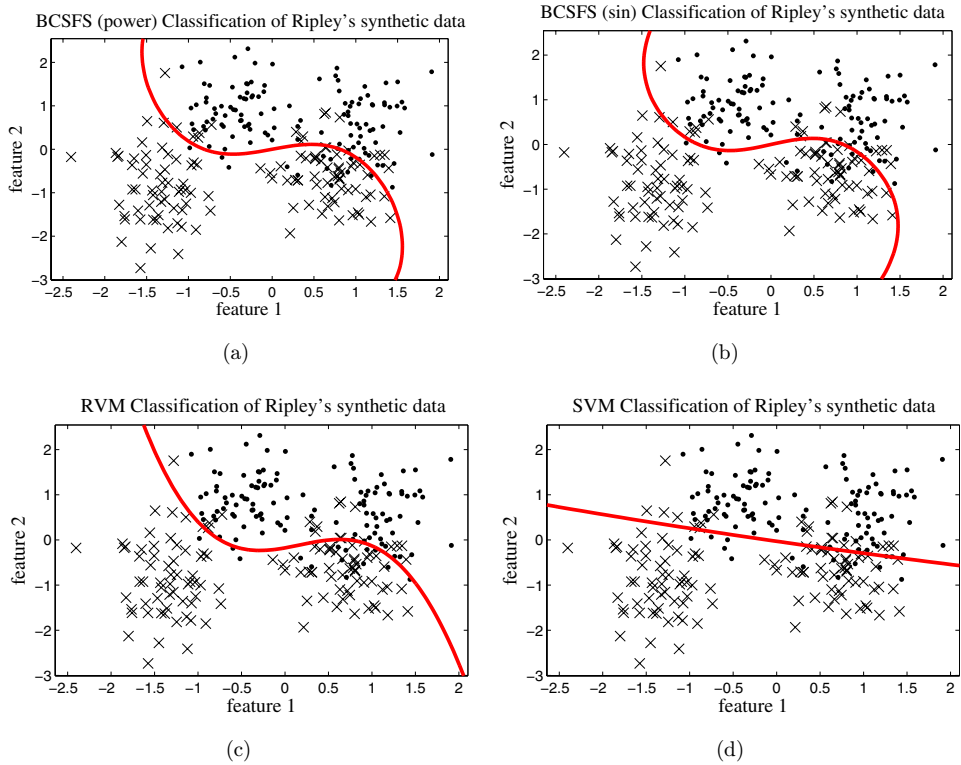


Fig. 2. (Color online) RVM, SVM and BCSFS via the VB approximation on the 250-example training set from the Ripley's synthetic data set, where the decision boundary is shown red. (a) BCSFS with power transformation, (b) BCSFS with sine transformation, (c) RVM with the Gaussian RBF kernel and (d) SVM with the Gaussian RBF kernel.

The synthetic data set used in this section is the Ripley's data set.³⁴ The following experiment is similar to that in Sec. 4.2 of Ref. 34. However, there are three differences about data pre-processing and data set usage in the experiment: (i) the data set was not normalized to have zero mean and unit variance in Ref. 34, while such normalization is utilized in the following experiment; (ii) a 100-example training set was randomly chosen from Ripley's original 250-example training set in Ref. 34, while we use the original training set in the following experiment; (iii) the feature selection problem was not considered in Ref. 34, while we add some noise features to validate the feature selection capability of our method in the following experiment. Here two versions of our BCSFS, i.e. BCSFS with power transformation and sine transformation, are compared with the RVM^b and SVM^c with the Gaussian RBF kernel, where the values of kernel parameters are selected via 5-fold cross-validation

^bThe toolbox of RVM is available at <http://www.relevancevector.com>.

^cThe toolbox of SVM is available at <http://asi.insa-rouen.fr/enseignants/~arakotom/toolbox/index.html>.

on the training set. For BCSFS, we set $a = b = c = d = 10^{-6}$, $e = 1$, $K = 100$, $M = 6$, and f is chosen via 5-fold cross-validation on the training set.

The learned classification boundaries for the 250-example training set (i.e. the given training set in the Ripley's data set) are given in Fig. 2. In Fig. 2, the decision boundaries of BCSFS and the RVM are similar, but that of the SVM tends to be linear, which is caused by the chosen kernel parameter via 5-fold cross-validation on the training set (here the kernel parameter is 12 for the SVM, and that is 3 for the RVM). For the feature selection, both the two forms of BCSFS select the original two features. For the selection of nonlinear transformations, BCSFS with power transformation and BCSFS with sine transformation select $m = 1, 3$ and $m = 1, 2$, respectively, which demonstrates that the proposed classifier can construct a simple decision boundary. The test error rate for the 1000-example test set (i.e. the given test set in the Ripley's data set) via the RVM (9.3%) is slightly superior to those via BCSFS (10.2% with power transformation and 10.3% with sine transformation), and that of the SVM (10.5%) is highest among the four classifiers.

In the following, we add four irrelevant Gaussian features, which do not contain any discriminative information between the original two classes, to the Ripley's synthetic data to validate the feature selection capability of BCSFS. The means and the standard deviations of the four Gaussian features are $\{0, 2\}$, $\{2, 1\}$, $\{1.5, 0.8\}$, and $\{0.5, 3\}$, respectively. Here the parameters which are needed to be adjusted are chosen by 5-fold cross-validation on the training set again.

After the training procedure, we obtain the selected features (the first feature and the second feature for both the two forms of BCSFS), the selected nonlinear transformations ($m = 1, 3$ for BCSFS with power transformation and $m = 1, 2$ for BCSFS with sine transformation), the decision boundaries (see Figs. 3(a) and 3(b)) and the

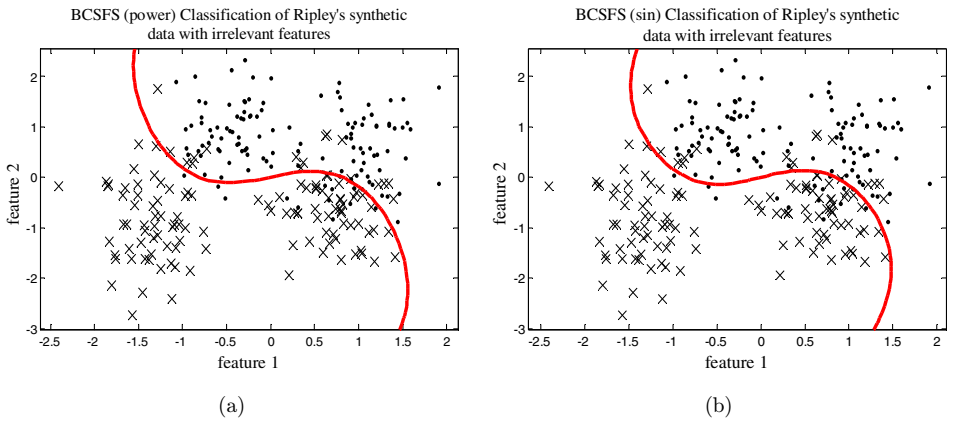


Fig. 3. (Color online) RVM, SVM and BCSFS via the VB approximation on the 250-example training set from the Ripley's synthetic data set with another four noise Gaussian features, where the decision boundary is shown red in the input-space with the original two features from the Ripley's synthetic data set. (a) BCSFS with power transformation, (b) BCSFS with sine transformation, (c) RVM with the Gaussian RBF kernel and (d) SVM with the Gaussian RBF kernel.

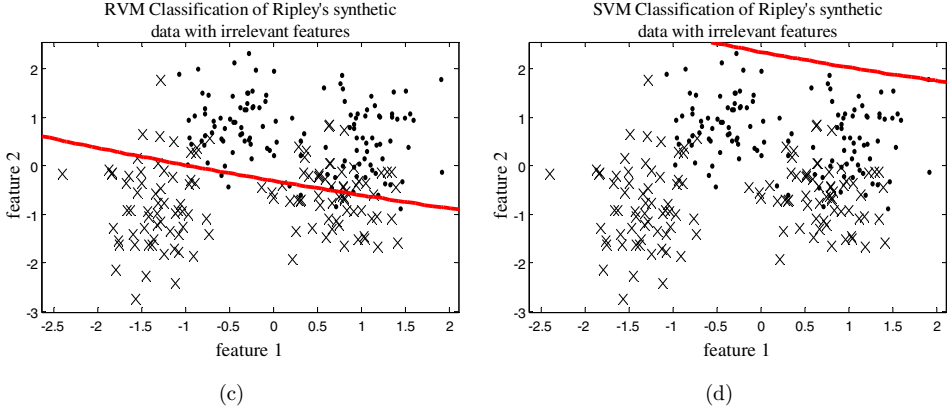


Fig. 3. (Continued)

test error rates (10.1% for BCSFS with power transformation and 10.3% for BCSFS with sine transformation) for BCSFS. Compared with Figs. 2(a) and 2(b), Figs. 3(a) and 3(b) show that the added four noise features hardly affect the classification decision boundaries of BCSFS. However, from Figs. 3(c) and 3(d) (here we only draw the decision boundary in the space of the original two features from the Ripley's synthetic data set), we can see that the added noise features greatly destroy the classification boundaries of the RVM and SVM (compared with those in Figs. 2(c) and 2(d)). And the test error rates are 10.9% and 10.7% for the RVM and SVM, respectively, which are higher than their original results without the noise features and those of the two versions of BCSFS.

With the original Ripley's synthetic data, our results are close to those of the RVM and the SVM, which means that BCSFS is comparable with the RVM and the SVM without noise features. The results obtained with adding four irrelevant features show that BCSFS is immune to the noise features. The feature selection ability

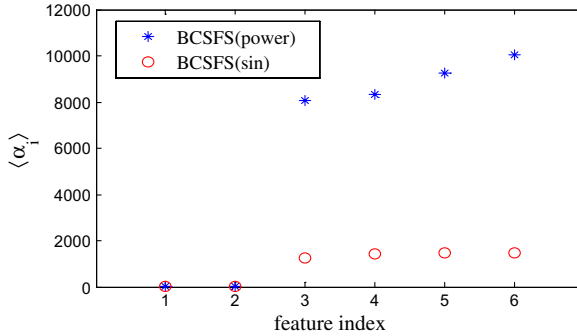


Fig. 4. $\langle \alpha_i \rangle$ inferred via BCSFS with power transformation and BCSFS with sine transformation for the 250-example training set from the Ripley's synthetic data set with another four noise Gaussian features, where the first two features are the original features in the Ripley's synthetic data set.

of BCSFS is clearly demonstrated in this experiment: among all the six features, BCSFS identifies only the two relevant features by choosing the smaller $\langle \alpha_i \rangle$ (the $\{\langle \alpha_i \rangle\}_{i=1}^P$ inferred via the two versions of BCSFS are shown in Fig. 4). The performance degradation and the great change of the decision boundary of the other two kernel methods show that the feature selection step is crucial for classification problem.

In the aforementioned part, we mainly analyze the feature selection capability of our method and demonstrate the importance of the feature selection. Now we will compare the classification accuracy and robustness of each method. Here the adopted data is the Ripley’s synthetic data with the four added irrelevant features used in Fig. 3. The training samples are randomly selected from the Ripley’s 250-example training set. The number of the training samples increases from 100 to 250 (the interval is 15). The Ripley’s 1000-example test set is fixed. Given the number of the training samples (100–235), the reported results are obtained by averaging over 20 random selections of the training set from all of the 250 training samples. The reported results for 250-example training set are the same as those given in the above paragraphs. The average error rates of the competing algorithms are compared in Fig. 5, which show that BCSFSs (including power transformation and sine transformation versions) perform better than the RVM and SVM, and are much more robust to the change of training data numbers.

Since the parameter inference via the full MCMC Gibbs sampling method is computationally expensive. In this paper, we employ the VB inference as a compromise between accuracy and efficiency. Following, we will present the experimental results of BCSFS on the performance of the VB approximation against the full Gibbs sampling method. Here we also utilize the original Ripley’s data set to evaluate the classification performance and CPU times of the two comparative inference methods.

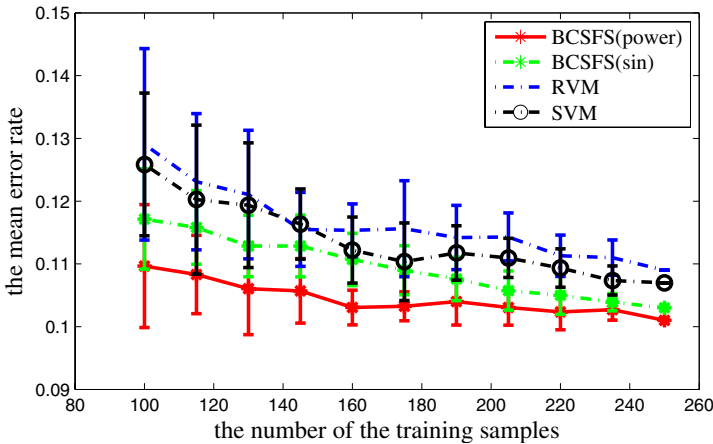


Fig. 5. The mean error rates of RVM, SVM, BCSFS with power transformation, and BCSFS with sine transformation versus the number of the training samples based on the Ripley’s synthetic data with the four added noise features. The change trendlines demonstrate the mean error rate and the error bars denote the corresponding standard deviation.

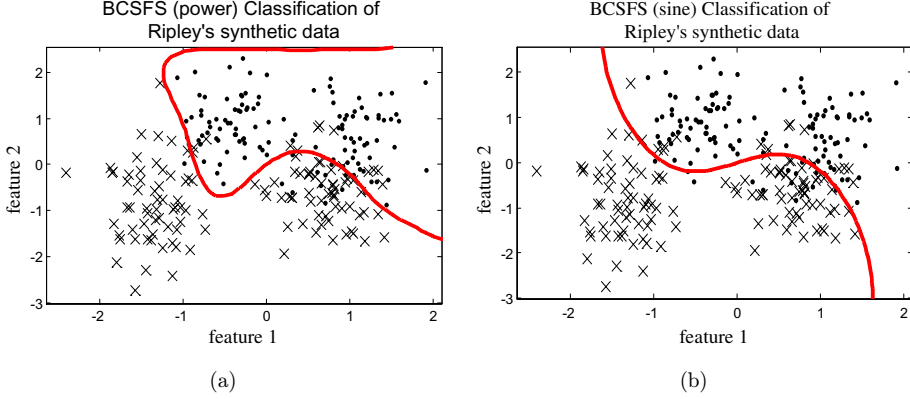


Fig. 6. (Color online) BCSFS via the Gibbs sampling method on the 250-example training set from the Ripley's synthetic data set, where the decision boundary is shown red. (a) BCSFS with power transformation and (b) BCSFS with sine transformation.

Table 1. CPU time (s) comparison for 2500 Gibbs samples versus 100 VB iterations.

Gibbs Sampling	VB
166.848 (s)	1.124 (s)

In Fig. 2, we have plotted the decision boundaries from the VB approximation after 100 iterations of BCSFS with power transformation and sine transformation. In Fig. 6, the decision boundaries produced from the full Gibbs sampling after 2500 samples of BCSFS with the same two transformations are also given. And the test error rates for the 1000-example test set via BCSFS with Gibbs sampling are 10.1% (power transformation) and 10.4% (sine transformation). As it can be seen, both the VB approximation and the Gibbs sampling obtain the similar decision boundaries and classification rates.

The CPU times of the two inference methods of BCSFS are given in Table 1. From Table 1, we can see that the full Gibbs sampling method is more time consuming than the VB approximation.

Therefore, from the above experimental results, we can see that VB approximation can achieve comparable recognition accuracy to the full Gibbs sampling in a shorter time.

4.2. Results on measured radar data set

The measured radar data set includes two classes: car and airplane, and each sample has 21 features. We randomly split this data set into 276 training samples and 274 test samples. To control the variability caused by random partitions, the splitting work is repeated 20 times and the results are averaged. In this experiment, BCSFS is compared against Feature Selection with Relevance Vector Machine (FSRVM^d),²⁵

^dThe software of FSRVM is obtained by directly revising the downloaded RVM toolbox.

Feature Generating Machine (FGM^e),³¹ RVM, SVM, and JCFO.^f For FGM, the number of the selected features, which should be fixed in advance, is chosen from $\{0.2P, 0.4P, 0.5P, 0.6P, 0.7P, P\}$ (where P is the total number of features). The RVM and SVM use the Gaussian RBF kernel, and JCFO uses both the r th order polynomial function kernel and the Gaussian RBF kernel, where the kernel parameters are chosen via 5-fold cross-validation on the training set. For BCSFS, two forms of nonlinear transformation are utilized: the power transformation and the exponent transformation, we set $a = b = c = d = 10^{-6}$, $e = 1$, and $K = 100$, and choose the parameters f and M via 5-fold cross-validation on the training set. Since BCSFS runs on the nonlinear transformed space, to compare BCSFS and other methods in the same transformed space fairly, we add two comparable algorithms: FSRVM with power transformation and FSRVM with exponent transformation. For the two direct nonlinear transformations of FSRVM, the parameter m is chosen by the exhaustive research method (corresponding to the optimal selection).

As mentioned above, a series of 20 trials is conducted. For BCSFS, in each trial, a feature, which is selected or not, is determined by the value of $\langle\alpha_i\rangle$, where the feature with the smaller $\langle\alpha_i\rangle$ (i.e. the larger $1/\langle\alpha_i\rangle$) is selected. Thus, we can obtain a feature-selection vector $[1/\langle\alpha_1\rangle, 1/\langle\alpha_2\rangle, \dots, 1/\langle\alpha_P\rangle]$ during each trial, which is unitary processed according to maximum value. In order to illustrate the robustness on feature selection via BCSFS, we can evaluate the mean and the standard deviation of every element (i.e. $1/\langle\alpha_i\rangle$ corresponding to the i th feature) in the normalized feature-selection vectors across the 20 trials. By setting a threshold (here we use 0.01), we can obtain the number of selected features via the average feature-selection vector, which may be different from the average value of the numbers of selected features via each single feature-selection vector. The larger the difference is, the less robust the feature selection of the method is. Similarly, for other competitive feature selection methods, there is also a parameter to determine a feature be selected or not. Therefore, for these methods, we can also evaluate the mean and standard deviation of each element in the normalized feature-selection vectors across the 20 trials. Through the same operation, for these methods, we can obtain the numbers of selected features via the average feature-selection vector and the average values of the numbers of selected features via each single feature-selection vector.

Table 2 provides the performances of feature selection and classification obtained by these competitive methods, where the boldface represents the minimum error rate of all methods for the given data set, and the symbol “\” represents that the method cannot select features. From Table 2, we can see that the numbers of the features selected by the two forms of BCSFS are smaller than those of the FSRVM, FGM, and the two forms of JCFO, which is true not only for the number of selected features via the average feature-selection vector but also for the average value of the numbers of selected features via each single feature-selection vector. Compared with FSRVM,

^eThe software of FGM is provided by the first author of Ref. 31.

^fThe software of JCFO is provided by the first author of Ref. 24.

Table 2. The numbers of selected features via the average feature-selection vector and the average standard deviations of all elements in the feature-selection vectors across the 20 trials, the average values of the numbers of selected features via each single feature-selection vector, the mean test errors, and the standard deviations of the test errors on the measured radar data are given, which are statistically computed over 20 runs from different partitions of training and test samples.

Method	Number of Selected Features via the Average Feature-selection Vector/the Average Standard Deviation of all Elements in the Feature-Selection Vectors	Average Value of the Numbers of Selected Features via Each Single Feature-Selection Vector	Mean Error Rate (%) / the Standard Deviation
BCSFS (power)	3/0.0070	3.3	3.03/0.88
BCSFS (exponent)	6/0.0450	4.5	2.88 /0.81
FSRVM	11/0.1019	7.85	3.21/1.05
FSRVM (power)	7/0.0634	6.35	3.47/1.19
FSRVM (exponent)	10/0.0990	8.6	3.01/1.12
FGM	21/0.1576	14	4.54/1.01
RVM (RBF)	\	\	3.76/0.83
SVM (RBF)	\	\	3.18/0.85
JCFO (poly)	19/0.0758	18.1	3.32/0.77
JCFO (RBF)	15/0.0611	10.4	4.67/1.16

FSRVM (exponent), FGM, and JCFO (RBF), the numbers of selected features of the two versions are much closer via our BCSFS methods. In addition, both the two forms of BCSFS possess the smaller standard deviations in the feature-selection vectors across the 20 trials. Therefore, from these results, we can say the feature selection capability of BCSFS is aggressive and robust. About the classification performance, BCSFSs (both the power transformation and the exponent transformation) are comparable to all other methods on the mean error rate and the standard deviation of the test errors.

Table 2 shows the aggressive and robust feature selection ability of BCSFS through the quantized form. In order to visually illustrate the feature selection

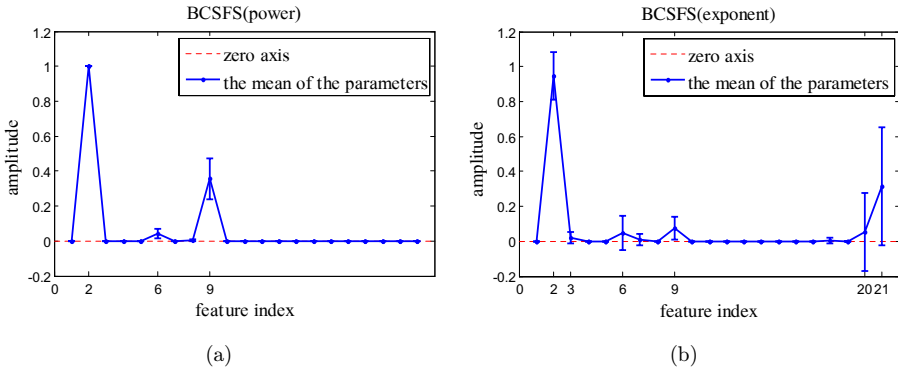


Fig. 7. The mean feature-selection parameters of BCSFS with power transformation, BCSFS with exponent transformation, FSRVM, FSRVM (exponent), FGM and JCFO (poly) for the 20 trials versus the corresponding features. The change trendline demonstrates the mean feature-selected parameters and the error bar denotes the corresponding standard deviation. The indices of the selected features are displayed in the horizontal coordinate for each method.

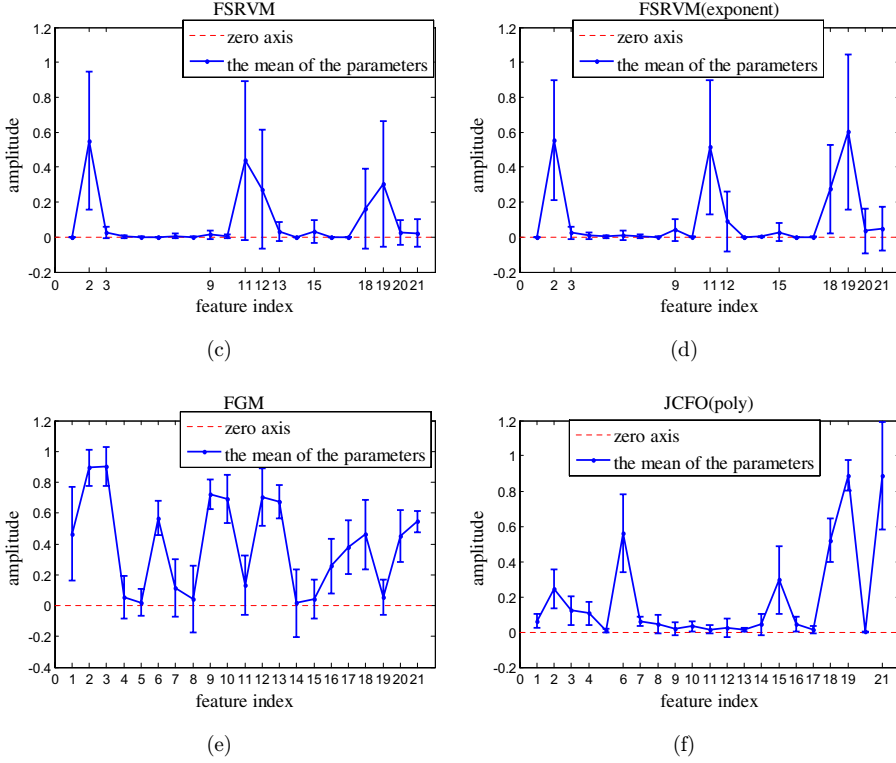


Fig. 7. (Continued)

capability of BCSFS, we display the mean and the standard deviation of every element in the normalized feature-selection vectors across the 20 trials through the graphic representation in Fig. 7. Similarly, for FSRVM, FSRVM (exponent), FGM and JCFO (poly), we can also figure their mean feature-selection parameters and the corresponding standard deviations in Fig. 7, where the indices of the selected features are displayed in the horizontal coordinate for each method. As shown in Fig. 7, the three features selected by BCSFS with power transformation are included in the features selected by BCSFS with exponent transformation.

4.3. Results on high-dimensional gene expression data set

To further illustrate the feature selection ability of our method, here we make an analysis on the colon data set,[§] which contains expression levels of 2000 genes from 40 tumor and 22 normal colon tissues. As discussed in Ref. 24, for such gene expression data set containing a large number of irrelevant and redundant variables, efficient feature selection is much more important than learning the classifier.

[§]The data set is available at <http://microarray.princeton.edu/oncology/affydata/index.html>.

To access diagnostic accuracy, we use a full leave-one-out cross-validation (LOOCV) procedure²⁴: we train on $k - 1$ samples and test the obtained classifier on the remaining sample, and repeat this procedure for every sample, then the k results are averaged.

Firstly, we verify the performance of BCSFS on all of the 2000 genes. Note, JCFO cannot be used due to its high computational cost. Therefore, we only consider the comparison between BCSFS and FGM, of which FGM shows great scalability to nonmonotonic feature selection on large-scale and very high-dimensional data sets.³¹ Due to the high feature dimensionality of the data set, we use the power transformation with small M in this experiment. The obtained accuracy is 85.48%, and the indices of the selected six genes are 377, 14, 356, 353, 1378, and 1772. To be fair, we set the number of features selected by FGM 6, which is equivalent to that of BCSFS. The classification accuracy of FGM on this data set is 85.48%, and the indices of the selected genes are 765, 377, 249, 1582, 1870, and 1772. From the results we can see that two selected features out of six are the same for BCSFS and FGM, in addition, BCSFS can obtain comparable recognition result with FGM. Therefore, we can say that BCSFS is also efficient when dealing with very high-dimensional problems.

To compare against other classification methods, we set another experiment same as that used in Ref. 24: we first apply a feature pre-selection step using some alternative approaches on the original data set, which can be efficiently implemented without matrix inversions (for example JCFO in a nonkernelized mode or the sparse probit regression (SPR) algorithm used in Ref. 27), and then adopt the full LOOCV procedure with a classification method on the pre-selected gene data. After the pre-selection, 42 genes are remained, which are same as those used in Ref. 24.

Our results are reported in Table 3, where BCSFS with exponent transformation and BCSFS with sine transformation are compared against some classification

Table 3. The numbers of selected features via the average feature-selection vector and the average standard deviations of all elements in the feature-selection vectors across the 62 trials, the average values of the numbers of selected features via each single feature-selection vector and accuracies on colon gene expression data set are given, which are statistically computed over the 62 runs in the LOOCV for BCSFS, FGM, and FSRVM, and those of JCFO are directly quoted from Refs. 24 and 22.

Method	Number of Selected Features via the Average Feature-Selection Vector/the Average Standard Deviation of all Elements in the Feature-Selection Vectors	Average Value of the Numbers of Selected Features via each Single Feature-Selection Vector	Accuracy (%)
BCSFS (exponent)	5/0.0272	5.89	88.70
BCSFS (sine)	5/0.0145	4.45	85.50
FSRVM	7/0.0267	7.11	88.70
FSRVM (exponent)	10/0.0278	8.56	88.70
FSRVM (sine)	9/0.0405	7.17	82.30
FGM	31/0.1421	20	88.70
JCFO (Linear kernel) ²⁴	26 ²²	26 ²²	96.80 ²⁴
JCFO (Quadratic kernel) ²⁴	26 ²²	26 ²²	88.70 ²⁴

methods with the feature selection ability, including FSRVM, FSRVM with exponent transformation, FSRVM with sine transformation, FGM, and JCFO (in Sec. 4.2 of Ref. 24, JCFO is superior to other classification methods used in Ref. 24, therefore, here we only give the classification accuracy of JCFO reported in Ref. 24). For BCSFS and FSRVM, the parameter settings are similar to that in the experiment of measurement radar data set. For FGM, the number of the selected features is chosen from $\{2, 5, 10, 15, 10\}$.

As in the experiment of radar data set, we can also evaluate the numbers of selected features via the average feature-selection vector and the average standard deviations of all elements in the feature-selection vectors for BCSFS (exponent transformation and sine transformation), FSRVM, FSRVM (exponent and sine), and FGM in Table 3. Besides the two results, we also provide the average values of the numbers of selected features via each single feature-selection vector and the accuracies for these methods in Table 3. From Table 3, we can see that the numbers of selected features of the two versions are much closer via our BCSFS methods, and both the two forms of BCSFS possess the fewer features and smaller average standard deviations in the feature-selection vectors compared with other methods. Therefore, BCSFS deserves the aggressive and robust feature selection capability. In addition, expect for the JCFO with linear kernel, BCSFS can also achieve comparable recognition accuracy to other methods.

Following, the means and the standard deviations of every element in the normalized (unitary processed according to the maximum value) feature-selected vectors (the features corresponding to the large parameters are selected) across the 62 trials for BCSFS (exponent transformation and sine transformation), FSRVM, FSRVM (exponent), and FGM are also drawn in Fig. 8, to illustrate their feature selection robustness visually. The genes that the JCFO algorithm associates with

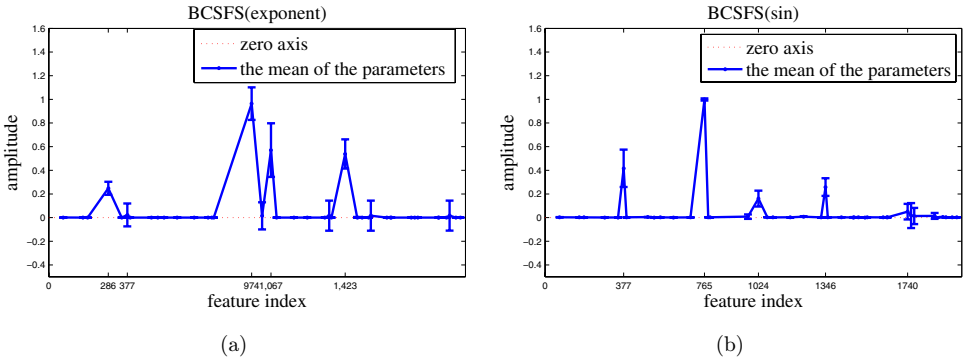


Fig. 8. (a)–(e) show the mean feature-selection parameters of BCSFS with exponent transformation, BCSFS with sine transformation, FSRVM, FSRVM (exponent), and FGM for 62 trials versus the corresponding features, respectively. The change trendline demonstrates the mean feature-selected parameters and the error bar denotes the corresponding standard deviation. (f) shows the normalized mean feature-selection parameters of JCFO given in Ref. 22 versus the corresponding features. The indices of the selected features are displayed in the horizontal coordinate for other methods.

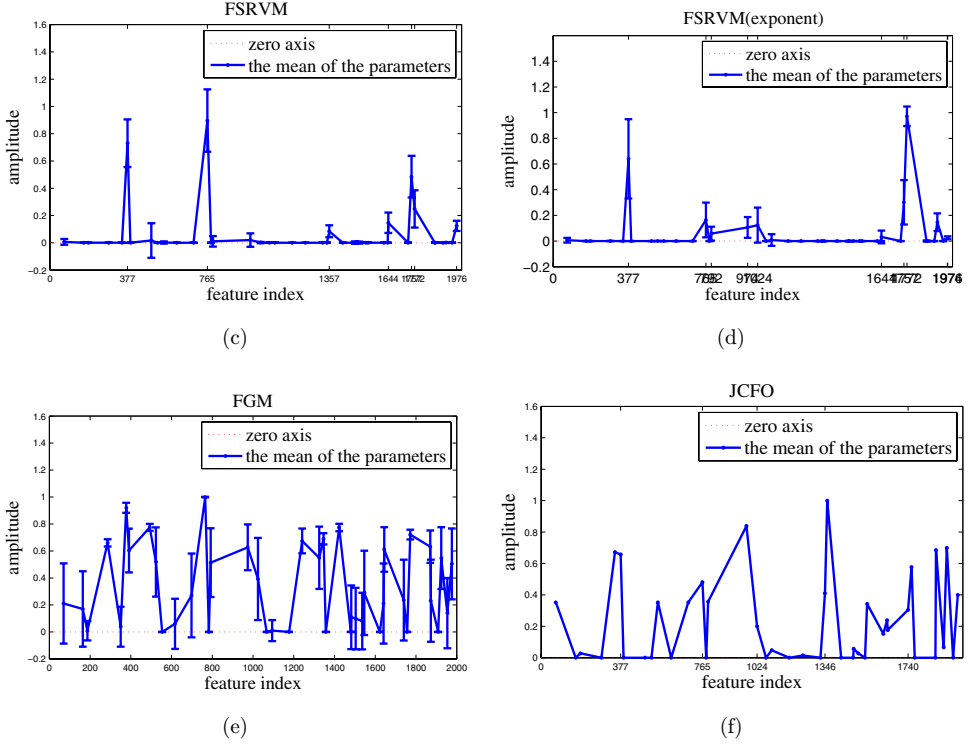


Fig. 8. (Continued)

high feature-selected parameter values of the colon data set are described in the medical literature.²² Since Ref. 22 only gives the indices of the selected features and corresponding average feature-selection parameter values through JCFO, we only draw the normalized (unitary processed according to the maximum value) means of each element of the feature-selection parameter without the corresponding standard deviations in Fig. 8(f). As shown in Fig. 8, the indices of the most important genes identified via our BCSFS with sine transformation are 765, 377, 1346, 1024, and 1740, which are also selected via JCFO and to be diagnosed importantly in Ref. 22.

4.4. Results on benchmark data sets

In this section, we use seven well-known benchmark data sets^h to compare our method with other methods. The details of seven data sets are shown in Table 4, among which the Monks3 and Hill-Valley-with-noise data sets have been originally divided into a training set and a test set. We manually partition other five data sets as described in Table 4.

^hThe data sets *Monks3*, *Wdbc*, *BCW*, *Australian*, *Sonar*, and *HVWN* are available at <http://archive.ics.uci.edu/ml/datasets.html>, and the data set *Crabs* is available at <http://www.stats.ox.ac.uk/pub/PRNN/>.

Table 4. Summary of selected data sets.

Data Set	Class	Training Size	Testing Size	Dimension
Monks3	2	122	310	6
Wdbc	2	285	284	30
Breast-cancer-wisconsin (BCW)	2	300	383	9
Australian	2	270	420	14
Crabs	2	80	120	5
Sonar	2	140	68	60
Hill-Valley-with-noise (HVWN)	2	606	606	100

Table 5. Numbers of selected features on seven benchmark data sets. The results are averaged over 20 runs from different partitions of training and test samples. The symbol “\” represents that the method cannot select features.

Method	Average Value of the Numbers of Selected Features via each Single Feature-Selection Vector						
	Monks3 (Features #: 6)	Wdbc (Features #: 30)	Crabs (Features #: 5)	BCW (Features #: 9)	Australian (Features #: 14)	Sonar (Feature #: 60)	HVWN (Feature #: 100)
BCSFS (power)	3	4.75	2	3.95	4.1	8.55	10
BCSFS (sine)	2	4.35	2	4.05	5.85	10.9	12
FSRVM	2	7.8	2.3	6.5	7.1	18.2	23
FSRVM (power)	2	12.95	3.2	6.15	5.15	12.6	27
FSRVM (sine)	3	7.7	2.35	6.65	7.6	18.4	34
FGM	3	12	5	5	9	36	40
RVM (RBF)	\	\	\	\	\	\	\
SVM (RBF)	\	\	\	\	\	\	\
JCFO (poly)	3	19.8	2.75	7.8	12.45	53.7	86

The RVM and SVM use the Gaussian RBF kernel, JCFO uses the r th-order polynomial function kernel, and the FSRVM adopts the no kernel function form²⁵ and the two nonlinear transformations: power transformation and sine transformation. Each classifier’s parameter setting is similar to that used in the experiment of radar data set.

Table 5 reports the average values of the numbers of selected features via each single feature-selection vector achieved by BCSFS and several other techniques, and Table 6 gives the mean test errors and the standard deviations of the test errors, where the results of the Wdbc, Crabs, BCW, Australian, and Sonar data sets are averaged over 20 random partitions. For our method, both the power transformation and sine transformation are used for all seven data sets. As in Table 5, the symbol “\” also represents the method cannot select features, and in Table 6, the boldface represents the minimum error rate of all methods for a given data set. For the Wdbc, Crabs, BCW, Australian, and Sonar data sets, both of two forms of the BCSFS have the comparable or even lower mean error rates comparing with other methods. On the Monks3 and HVWN data sets, BCSFS with power transformation performs nearly as well as the best available approach about the mean error rate. In addition,

Table 6. Mean test errors and standard deviation of the test errors on seven benchmark data sets. The results are averaged over 20 runs from different partitions of training and test samples.

Method	Mean Error Rate (%) / the Standard Deviation						
	Monks3	Wdbc	Crabs	BCW	Australian	Sonar	HVWN
BCSFS (power)	2.58	3.33/0.84	2.13/1.36	3.25/0.55	13.64/1.20	18.60/4.56	13.20
BCSFS (sine)	18.06	3.29/0.86	3.75/1.25	3.15/0.77	13.65/1.11	23.46/4.59	23.76
FSRVM	19.35	3.86/1.30	4.00/1.03	3.52/0.85	13.86/1.14	25.81/4.64	13.20
FSRVM (power)	18.06	17.98/2.02	5.33/1.63	4.10/1.08	13.98/0.61	34.34/4.80	36.96
FSRVM (sine)	19.35	3.71/1.02	4.04/1.06	3.38/0.84	14.02/0.06	23.82/4.95	19.14
FGM	14.19	3.27/0.97	4.62/1.44	3.25/0.65	13.99/1.30	22.21/3.81	32.01
RVM (RBF)	2.90	3.71/0.96	3.63/1.58	3.42/0.83	14.43/1.24	19.19/4.59	10.40
SVM (RBF)	2.58	3.56/1.29	4.25/1.71	3.29/0.64	14.44/1.03	19.26/5.64	33.17
JCFO (poly)	0	3.86/1.16	3.87/1.09	3.49/0.94	14.02/0.96	18.60/3.87	24.42

if the data sets are of relatively low feature dimensionality (e.g. the Monks3 and Crabs), there is no great difference for selecting features among BCSFS, FSRVM, FGM, and JCFO. However, for the relatively high-dimensional data sets, we note that BCSFS (power transformation) only selects 4.75, 3.95, 4.1, 8.55, and 10 features, for the Wdbc, BCW, Australian, Sonar, and HVWN data sets, respectively, while JCFO selects 19.8, 7.8, 12.45, 53.7, and 86 features for the five data sets, respectively.

The above experiments show the classification results obtained by our algorithm are not worse or even better than those of other methods for each data set. Although all the methods are competitive in terms of error rates on these data sets, however, compared with the RVM and SVM, our method can select features simultaneously. Three of the competing methods (FSRVM, FGM, and JCFO) also have the ability to perform feature selection, but BCSFS can select fewer features from the relatively high-dimensional data. In addition, even in the same nonlinear transformed space, FSRVM with nonlinear transformations perform worse than our method in both the feature selection and classification performance. This implies that the extension of the nonlinear transformation and the sparse selection of the nonlinear transformations play important roles in our method.

4.5. Analysis of the algorithm complexity

In the training stage, the update rules for the parameters depend on computing the posterior weight covariance matrix $\tilde{\Sigma}_w$ in Eq. (15), which requires an inverse operation (in fact Cholesky decomposition) of order $O((MP)^3)$ complexity, with M denoting the number of nonlinear mappings and P denoting the number of features. Since the computation cost in the training phase can be ignored for an off-line learning system in the practical applications, it is more important to evaluate the computation complexity during the test procedure. Here we mainly compare our method with several kernel methods including the RVM, SVM, and JCFO. The exponent transformation used in BCSFS is comparable to the Gaussian RBF kernel used in the other three kernel methods. And the power transformation used in

Table 7. The algorithm complexity of BCSFS, RVM, SVM, and JCFO during the test procedure, where A denoting the number of selected features, B denoting the number of selected exponent transformations, X denoting the number of relevance/support vectors, P denoting the numbers of original features, $L(\langle z_m \rangle) \in \{0, 1\}$ denoting the label of $\langle z_m \rangle$ calculated in the training phase and r denoting the order of the polynomial kernel.

Methods	Algorithm Complexity with the Exponent Transformation or the Gaussian RBF Kernels	Algorithm Complexity with the Power Transformation or the r th Order Polynomial Function Kernel
BCSFS	$O(AB)$	$O(A \sum_{m=1}^M mL(\langle z_m \rangle))$
RVM	$O(XP)$	$O(XPr)$
SVM	$O(XP)$	$O(XPr)$
JCFO	$O(XA)$	$O(XAr)$

BCSFS is comparable to the r th-order polynomial function kernel used in other kernel methods. Table 7 shows the algorithm complexity of the four methods during the test procedure, where A is the number of selected features; B represents the number of selected exponent transformations; X denotes the number of relevance/support vectors; P is the numbers of original features; $L(\langle z_m \rangle) \in \{0, 1\}$ is the label of $\langle z_m \rangle$ calculated in the training phase; and r represents the order of the polynomial kernel.

Now, we give an example with the measured radar data set based on 21 measured variables to compare the complexity of these methods. For BCSFS, as shown in Table 2, the numbers of selected features and selected exponent transformations are 4.5 and 4, respectively, therefore, its complexity is of order $O(18)$; for the RVM and SVM, the numbers of the relevance vectors and support vectors are 6.75 and 35.6, respectively, therefore, the complexities are of order $O(141.75)$ and $O(747.6)$; for JCFO, the numbers of the selected features and relevance vectors are 10.4 and 8, respectively, therefore, its complexity is of order $O(83.2)$. The results show that the complexity of BCSFS is further smaller than those of the RVM, SVM and JCFO on this measured radar data set.

5. Conclusion

A BCSFS algorithm which joins classification and nonlinear feature selection together via nonlinear mappings is proposed in this paper. In addition to using α_p to select the relevant features, BCSFS also uses the Beta process prior to encourage many of $\{z_m\}_{m=1}^M$ to be zero, which can remove the irrelevant nonlinear transformations. The experiments on both low-dimensional and high-dimensional data sets show that our proposed BCSFS possesses more aggressive feature selection capability than other competitive methods with comparable or even lower mean error rare.

However, our current work also has some limitations: (1) In this paper, we present three commonly used nonlinear mapping functions and the linear mapping function (the power transformation with $M = 1$). Nevertheless, there is not a conclusion how to choose the optimal mapping function for a given data set. For some given

problems, we still need to further analyze which mapping function is optimal in the future work. (2) In this paper, the mapping function of the input sample should be extended M times ($M \geq 1$). In fact the value of M cannot be infinitely large, since there may be some difficulty in the inverse operation for a large-scale matrix $\tilde{\Sigma}_w = (\text{diag}(\alpha) + (\Phi(\mathbf{X})^T \text{diag}(\tau) \Phi(\mathbf{X})) \odot (\langle \mathbf{z} \mathbf{z}^T \rangle))^{-1}$, especially for the power transformation. Therefore, we set $M = 6$ in the experiments. (3) In this paper, the nonlinear feature mapping works on each feature directly without considering the complex feature interactions. Therefore, we will improve our model to consider the complex feature interactions in the future work.

Acknowledgments

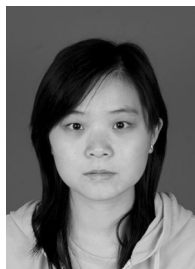
This work was partially supported by the National Natural Science Foundation of China (No. 61271024 and No. 61322103), and the Foundation for Author of National Excellent Doctoral Dissertation of P. R. China (FANEDD-201156). The authors would like to thank the editors and the anonymous reviewers for their valuable comments and suggestions. In addition, the JCFO and FGM codes are respectively provided by Dr. Krishnapuram and Dr. Tan for the comparison in our experiments.

References

1. T. Amemiya, Regression analysis when the dependent variable is truncated normal, *Econometrica* **41** (1973) 997–1017.
2. M. J. Beal, Variational algorithms for approximate Bayesian inference, Ph.D. Thesis, University of London, UK (2003).
3. C. M. Bishop, *Pattern Recognition and Machine Learning* (Springer, New York, 2006).
4. C. M. Bishop and M. E. Tipping, Variational relevance vector machines, in *Proc. 16th Conf. Uncertainty in Artificial Intelligence* (Morgan Kaufmann, San Francisco, USA, 2000), pp. 46–53.
5. P. S. Bradley and O. L. Mangasarian, Feature selection via concave minimization and support vector machine, in *Proc. 13th Int. Conf. Machine Learning*, San Francisco, CA (1998), pp. 82–90.
6. J. C. Christopher, Burges, A tutorial on support vector machine for pattern recognition, *Data Min. Knowl. Discov.* **2** (1998) 121–167.
7. T. Damoulas and M. A. Girolami, Probabilistic multi-class multi-kernel learning: On protein fold recognition and remote homology detection, *Bioinformatics* **24** (2008) 1264–1270.
8. T. Damoulas and M. A. Girolami, Pattern recognition with a Bayesian kernel combination machine, *Pattern Recogn. Lett.* **30** (2009) 46–54.
9. T. Damoulas and M. A. Girolami, Combining feature spaces for classification, *Pattern Recogn.* **42** (2009) 2671–2683.
10. T. Damoulas, Y. Ying, M. A. Girolami and C. Campbell, Inferring sparse kernel combinations and relevance vectors: An application to subcellular localization of proteins, in *Proc. 7th Int. Conf. Machine Learning and Applications (ICMLA)* (2008), pp. 577–582.
11. X. H. Ding, L. H. He and L. Carin, Bayesian robust principal component analysis, *IEEE Trans. Image Process.* **20** (2011) 3419–3430.

12. L. Du, M. H. Chen, J. Lucas and L. Carin, Sticky hidden markov modeling of comparative genomic hybridization, *IEEE Trans. Signal Process.* **58** (2010) 5353–5368.
13. L. Du, P. H. Wang, H. W. Liu, M. Pan, F. Chen and Z. Bao, Bayesian spatiotemporal multitask learning for radar HRRP target recognition, *IEEE Trans. Signal Process.* **59** (2011) 3182–3195.
14. M. A. T. Figueiredo, Adaptive sparseness for supervised learning, *IEEE Trans. Pattern Anal. Mach. Intell.* **25** (2003) 1150–1159.
15. I. K. Fodor, A survey of dimension reduction techniques, Technical report UCRL-ID-148494, center for applied scientific computing, Lawrence Livermore National Laboratory (2002).
16. J. J. Fuchs, On the application of the global matched filter to DOA estimation with uniform circular arrays, *IEEE Trans. Signal Process.* **49** (2001) 702–709.
17. G. M. Fung and O. L. Mangasarian, A feature selection newton method for support vector machine classification, *Comput. Optim. Appl.* **28** (2004) 185–202.
18. W. R. Gilks, S. Richardson and D. J. Spiegelhalter, *Markov Chain Monte Carlo in Practice* (Chapman & Hall, London, UK, 1996).
19. M. Girolami and S. Rogers, Variational Bayesian multinomial probit regression with Gaussian process priors, *Neural Comput.* **18**(8) (2006) 1790–1817.
20. S. H. Ji, D. Dunson and L. Carin, Multi-task compressive sensing, *IEEE Trans. Signal Process.* **57** (2009) 92–106.
21. K. Koh, S. J. Kim and S. Boyd, An interior-point method for large-scale l_1 -regularized logistic regression, *J. Mach. Learn. Res.* **8** (2007) 1519–1555.
22. B. Krishnapuram, L. Carin and A. J. Hartemink, Joint classifier and feature optimization for cancer diagnosis using gene expression data, in *Proc. 7th Ann. Int. Conf. Computational Molecular Biology (RECOMB)* (2003), pp. 167–175.
23. B. Krishnapuram, A. J. Hartemink and L. Carin, Logistic regression and RVM for cancer diagnosis from gene expression signatures, in *Proc. Workshop Genomic Signal Processing and Statistics (GENSIPS)* (2002).
24. B. Krishnapuram, A. J. Hartemink, L. Carin and M. A. Figueiredo, A Bayesian approach to joint feature selection and classifier design, *IEEE Trans. Pattern Anal. Mach. Intell.* **26**(9) (2004) 1105–1111.
25. D. F. Li and W. C. Hu, Feature selection with RVM and its application to prediction modeling, *Advances in Artificial Intelligence*, Vol. 4304 (2006), pp. 1140–1144.
26. L. B. Li, M. Y. Zhou, G. Sapiro and L. Carin, On the integration of topic modeling and dictionary learning, in *Proc. 28th Int. Conf. Machine Learning*, Bellevue, WA, USA (2011), pp. 625–632.
27. Z. Liu, F. Jiang, G. Tian *et al.*, Sparse logistic regression with L_p penalty for biomarker identification, *Stat. Appl. Genet. Mol. Biol.* **6** (2007) 1–22.
28. P. McCullagh and J. Nelder, *Generalized Linear Models* (Chapman and Hall, London, 1989).
29. J. Paisley and L. Carin, Nonparametric factor analysis with beta process priors, in *Proc. Int. Conf. Machine Learning* (2009), pp. 777–784.
30. I. Psorakis, T. Damoulas and M. A. Girolami, Multiclass relevance vector machines: Sparsity and accuracy, *IEEE Trans. Neural Netw.* **21** (2010) 1588–1598.
31. M. Tan, L. Wang and I. W. Tsang, Learning sparse SVM for feature selection on very high dimensional datasets, in *Proc. 27th Int. Conf. Machine Learning (ICML 2010)*, Haifa, Israel, June 2010, pp. 1047–1054.
32. R. Tibshirani, Regression shrinkage and selection via the lasso, *J. R. Stat. Soc. B* **58** (1996) 267–288.

33. R. Tibshirani, The lasso method for variable selection in the cox model, *Stat. Med.* **16** (1997) 385–395.
 34. M. E. Tipping, Sparse Bayesian learning and the relevance vector machine, *J. Mach. Learn. Res.* **1** (2001) 211–244.
 35. J. Weston, A. Elisseeff and B. Scholkopf, Use of the zero-norm with linear models and kernel methods, *J. Mach. Learn. Res.* **3** (2003) 1439–1461.
 36. J. Zhu, S. Rosset, T. Hastie and R. Tibshirani, 1-norm support vector machines, in *Neural Information Processing Systems 16* (MIT, R, Cambridge, 2004), pp. 49–56.
-



Danlei Xu received her B.S. and Ph.D. degrees in Electronic Engineering from Xidian University, Xi'an, China, in 2009 and 2015. Her research interests include radar signal processing, radar automatic target recognition (RATR), and machine learning.



Lan Du (M'11) received her B.S., M.S., and Ph.D. degrees in Electronic Engineering from Xidian University, Xi'an, China, in 2001, 2004, and 2007, respectively. Her doctoral dissertation was granted National Excellent Doctoral Dissertation of P. R. China in 2009.

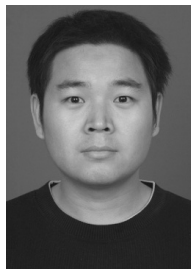
From September 2007 to September 2009, she did research work as a Postdoctoral Research Associate in the Department of Electrical and Computer Engineering, Duke University, Durham, NC 27708 USA. She is currently a Professor at the National Laboratory of Radar Signal Processing, Xidian University.

Her main research interests are in the fields of statistical signal processing and machine learning with application to radar target recognition.



Hongwei Liu (M'04) received his M.Eng. and Ph. D. degrees in Electronic Engineering from Xidian University, Xi'an, China, in 1995 and 1999, respectively. From 2001 to 2002, he was a visiting scholar with the Department of Electrical and Computer Engineering, Duke University, Durham, NC 27708 USA. He is currently a Professor at the National Laboratory of Radar Signal Processing, Xidian University, and the Director of this laboratory.

His research interests are radar signal processing, radar automatic target recognition (RATR), adaptive signal processing, and cognitive radar.



Penghui Wang received his B.S. degree in Communication Engineering from National University of Defense Technology (NUDT), Changsha, China, in 2005 and his Ph.D. in Signal Processing from Xidian University, Xi'an, China, in 2012. He is currently a

lecturer at the National Laboratory of Radar Signal Processing, Xidian University. His research interests include radar signal processing and automatic target recognition.

Copyright of International Journal of Pattern Recognition & Artificial Intelligence is the property of World Scientific Publishing Company and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.